

Calcolatori Elettronici

4 settembre 2025

Teoria

Esercizio 1 (7 punti): Si realizzi una rete sequenziale che riceve in ingresso una sequenza infinita di caratteri appartenenti all'alfabeto $I = \{a, b\}$ e riconosca tutte le occorrenze (anche sovrapposte) della sottostringa *abba*. In caso di riconoscimento, l'output della rete è *accettato*. In tutti gli altri casi, l'output è *non accettato*. Realizzare il circuito equivalente dell'equazione di eccitazione di un flip flop di stato.

Esercizio 2 (7 punti): Siano $A = -1X$ e $B = 3Y$ due numeri interi, in cui X e Y sono, rispettivamente, il penultimo ed ultimo numero della propria matricola—occorre *sostituire* le cifre della matricola a X e Y , non eseguire una moltiplicazione. Calcolare la rappresentazione binaria, in complemento a due a 8 bit, di tali valori. Eseguire l'operazione di somma tra i due numeri nella loro rappresentazione binaria, mostrando tutti i passaggi. Specificare se si verifica un overflow. Convertire il risultato finale in decimale e verificare la correttezza dell'operazione.

Esercizio 3 (8 punti): Convertire in C il seguente frammento di codice assembly.

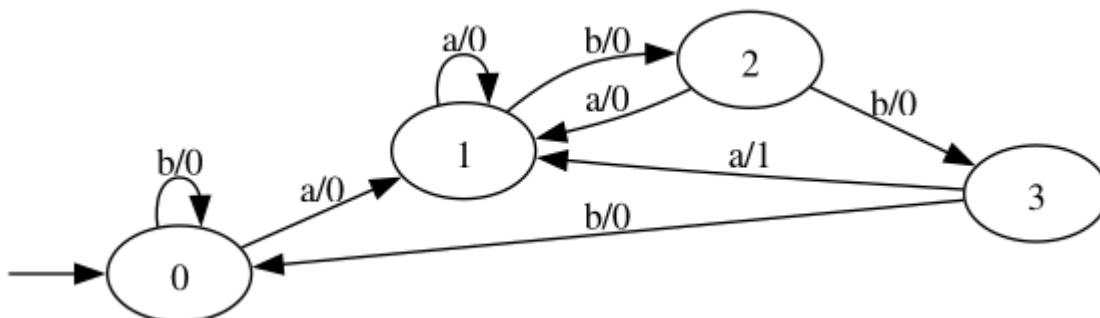
```
1 fun:
2     xor %edx, %edx
3     test %edi, %edi
4     jz .end
5     .loop:
6     lea -0x1(%edi), %eax
7     add $0x1, %edx
8     and %eax, %edi
9     jnz .loop
10    .end:
11    movl %edx, %eax
12    ret
```

Esercizio 4 (8 punti): Discutere il funzionamento del carry lookahead adder, illustrandone i vantaggi prestazionali rispetto al full adder.

Soluzioni

Esercizio 1

Codificando l'alfabeto di uscita come $O = \{\text{non accettato} : 0, \text{accettato} : 1\}$, si può ottenere il seguente automa a stati finiti che rappresenta la rete sequenziale richiesta:



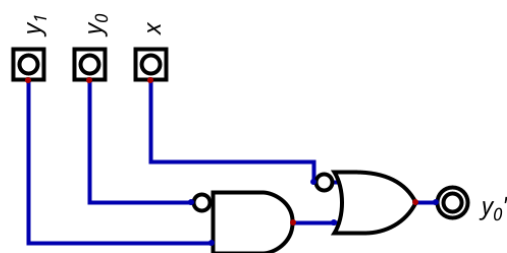
Codifichiamo gli stati con il corrispondente valore binario e l'alfabeto di input come $I = \{a : 0, b : 1\}$. Otteniamo la seguente tabella degli stati e transizioni.

y_1	y_0	x	y'_1	y'_0	z
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	0	1	0
1	0	1	1	1	0
1	1	0	0	1	1
1	1	1	0	0	0

Scegliendo di minimizzare l'equazione di eccitazione di y'_0 , si può procedere con la costruzione della seguente mappa di Karnaugh:

$y_1 y_0$ \ x	00	01	11	10
0	1	1	1	1
1	0	0	0	1

che porta all'equazione $y'_0 = \bar{x} + y_1 \bar{y}_0$, corrispondente al seguente circuito:



Esercizio 2

Si suppongono $x=0$ e $y=1$. Per convertire -10 partiamo dalla conversione in binario del valore positivo, ottenendo per divisioni successive $10_{10} = 00001010_2$. Ora, effettuiamo l'operazione di complemento a due, ottenendo $-10_{10} = 11110110_2$. Per convertire 31 è sufficiente procedere con le divisioni successive, ottenendo $31_{10} = 00011111_2$.

La somma si ottiene effettuando la somma bit a bit:

$$\begin{array}{r}
 1 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0 \\
 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 1 \\
 \hline
 0 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1
 \end{array}$$

Si noti che il riporto non è da prendere in considerazione, trattandosi di somma in complemento a due. Viceversa, poiché gli ultimi due riporti sono concordi (entrambi a 1), non vi è overflow. Ciò era prevedibile poiché, essendo gli operandi di segno opposto, il risultato è sempre rappresentabile. Infatti: $00010101 = 21_{10} = -10 + 31$.

Esercizio 3

La funzione utilizza il registro `%edi` senza averlo inizializzato, pertanto si capisce che accetta un parametro di tipo `int`. Dopodiché, il frammento assembly azzerà `%edx` che sarà quindi utilizzata come variabile d'appoggio, verifica se l'argomento in `%edi` è nullo e, in caso contrario, entra in un ciclo.

All'interno del ciclo, è interessante osservare l'utilizzo di `lea`: tale istruzione (load effective address), è stata progettata per valutare una modalità di indirizzamento senza effettuare alcun accesso a memoria. La `lea`, quindi, valuta l'espressione $base + indice \cdot scala + spiazamento$ e ne salva il risultato nel registro destinazione. In tale utilizzo, non siamo interessati a valutare un indirizzo, ma ci limitiamo a valutare un'espressione nel formato $base + indice \cdot scala + spiazamento$. Pertanto, tale istruzione equivale a $eax = edi - 1$, molto più compatta di una sequenza `movl %edi, %eax` seguita da `add $1, %eax`.

Il codice poi prosegue con l'incremento del contatore in `%edx` e aggiorna `%edi` con l'operazione `%edi &= %eax`. L'istruzione di salto condizionale testa il risultato dell'AND e ripete finché non diventa zero. Poiché la sequenza implementa la trasformazione iterativa $x \leftarrow x \& (x - 1)$, ogni iterazione cancella il bit a 1 meno significativo, e il numero totale di iterazioni coincide con il numero di bit a 1 dell'argomento. Al termine, il valore in `%edx` viene copiato in `%eax`, suggerendo quindi che il frammento di codice è una funzione che calcola il numero di bit impostati ad 1 in un valore intero.

Poiché quindi non è interessante il fatto che il parametro passato sia segnato/non segnato, possiamo ritornare sul ragionamento iniziale e dire che l'unico parametro della funzione è un `unsigned`. Il codice C equivalente, quindi, può essere il seguente:

```
1 unsigned count_bit_set(unsigned x)
2 {
3     unsigned int count = 0;
4
5     if (x == 0)
6         return 0;
7
8     do {
9         count += 1;
10        x &= x - 1;
11    } while (x != 0);
12
13    return count;
14 }
```

Progetto

Un processore `z64` governa una postazione di accettazione pacchi per la pre-classificazione alla spedizione. Il sistema impiega le seguenti periferiche.

- Una `BILANCIA` elettronica che, quando interrogata, fornisce il peso del collo come valore a 8 bit non segnato. L'unità di misura è l'ettogrammo, quindi il massimo peso rappresentabile è $255hg = 25,5kg$.
- Un `LETTORE` di codici a barre che genera una richiesta di interruzione all'arrivo di un nuovo pacco. `LETTORE` ha a disposizione un registro di interfaccia di dimensione longword, contenente i 4 byte del codice a barre.

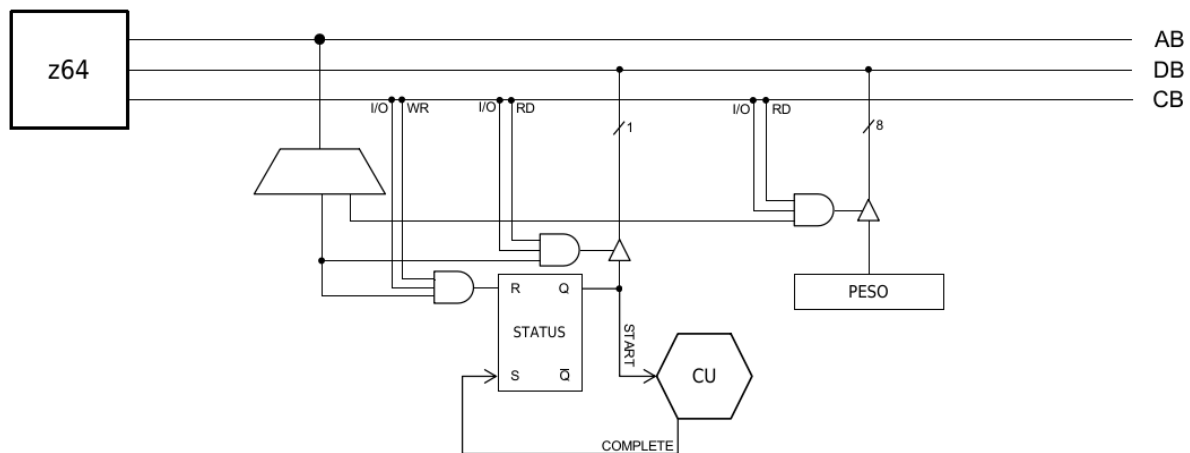
Dopo aver determinato la classe di costo, lo `z64` scrive in memoria, all'interno del vettore `tabella_costi`, un record composto dai 4 byte del codice a barre seguiti dal byte della classe di costo. Il vettore `tabella_costi` può mantenere fino a 128 record di 5 byte e viene utilizzato in maniera circolare: quando esso è pieno, si comincia a riscrivere dall'inizio del vettore.

Realizzare:

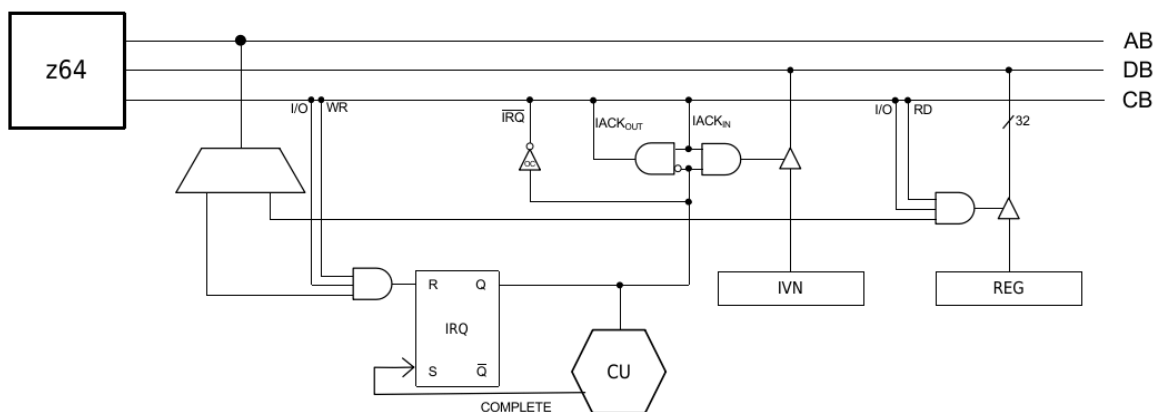
- Le interfacce dei dispositivi **BILANCIA** e **LETTORE** ;
- Tutto il codice necessario al funzionamento del sistema, senza l'utilizzo di pseudo-istruzioni.

Soluzione

BILANCIA è una periferica sincrona con un registro di input a 8 bit:



LETTORE è una periferica asincrona di output con un singolo registro di interfaccia per trasmettere il codice:



Una possibile implementazione del codice del sistema è la seguente

```

1  .org 0x800
2  .equ BILANCIA_STATUS, 0x10
3  .equ BILANCIA_PESO, 0x11
4  .equ LETTORE_IRQ, 0x12
5  .equ LETTORE_REG, 0x12
6  .data
7  classi_costo: .fill 27, 1
8  tabella_costi: .fill 128, 5
9  pos: .word 0
10 .text
11 main:
12     sti
13     .loop:
14     hlt
15     jmp .loop
16
17 .driver 0 # lettore
18     push %rax
19     push %rbx
20     push %rdi
21     push %rdx
22     push %rcx
23
24     # Recupera il codice a barre
25     inl $LETTORE_REG, %eax
26     movl %eax, %ebx
27
28     # Recupera il peso in ettogrammi
29     outb %al, $BILANCIA_STATUS
30     .bw:
31     inb $BILANCIA_STATUS, %al
32     btb $0, %al
33     jnc .bw
34     inb $BILANCIA_PESO, %al
35
36     # Conversione da etti a chili
37     movzbl %al, %edi
38     call etti_a_kg
39     movzbl %eax, %rcx
40
41     # Recupera la classe di costo
42     movzbl classi_costo(,%rcx,1), %edx
43
44     # Scrive il record richiesto nella tabella
45     movzwl pos, %rcx
46     movl %ebx, (%rdi) # 4 byte del codice a barre
47     movb %dl, 4(%rdi) # classe di costo
48
49     # Gestione circolare del buffer
50     addq $5, %rcx
51     cmpq $640, %rcx # 640 = 128 * 5
52     jnz .ok
53     xorq %rcx, %rcx
54     .ok:
55     movw %cx, pos
56
57     # Fine: si possono nuovamente ricevere richieste di interruzione
58     outb %al, $LETTORE_IRQ
59     pop %rcx
60     pop %rdx
61     pop %rdi
62     pop %rbx
63     pop %rax
64     iret
65
66     # Converte ettogrammi in chilogrammi arrotondati

```

```
67 # E un ciclo che calcola [(peso + 5) / 10]
68 etti_a_kg:
69     add $5, %edi
70     xor %ecx, %ecx
71     .Ldiv10:
72     cmp $10, %edi
73     jb  .Ldone
74     sub $10, %edi
75     inc %ecx
76     jmp .Ldiv10
77     .Ldone:
78     mov %ecx, %eax
79     ret
```